
python
django

What is Django

- Emphasize Django as a batteries-included web framework.
- It takes care of tasks like database handling, user authentication, and routing so developers can focus on features instead of boilerplate code.
- It comes with many built-in features like user authentication, an admin panel and form handling.

Django Architecture

- Django is based on MVT (Model-View-Template) architecture.
- MVT is a software design pattern for developing a web application.

1.Model: Acts as the data interface. It defines the structure of your data and is usually backed by a database (e.g., MySQL, PostgreSQL).

2.View: A Python function or class that handles web requests. It interacts with the Model and renders a response, typically by passing data to a Template.

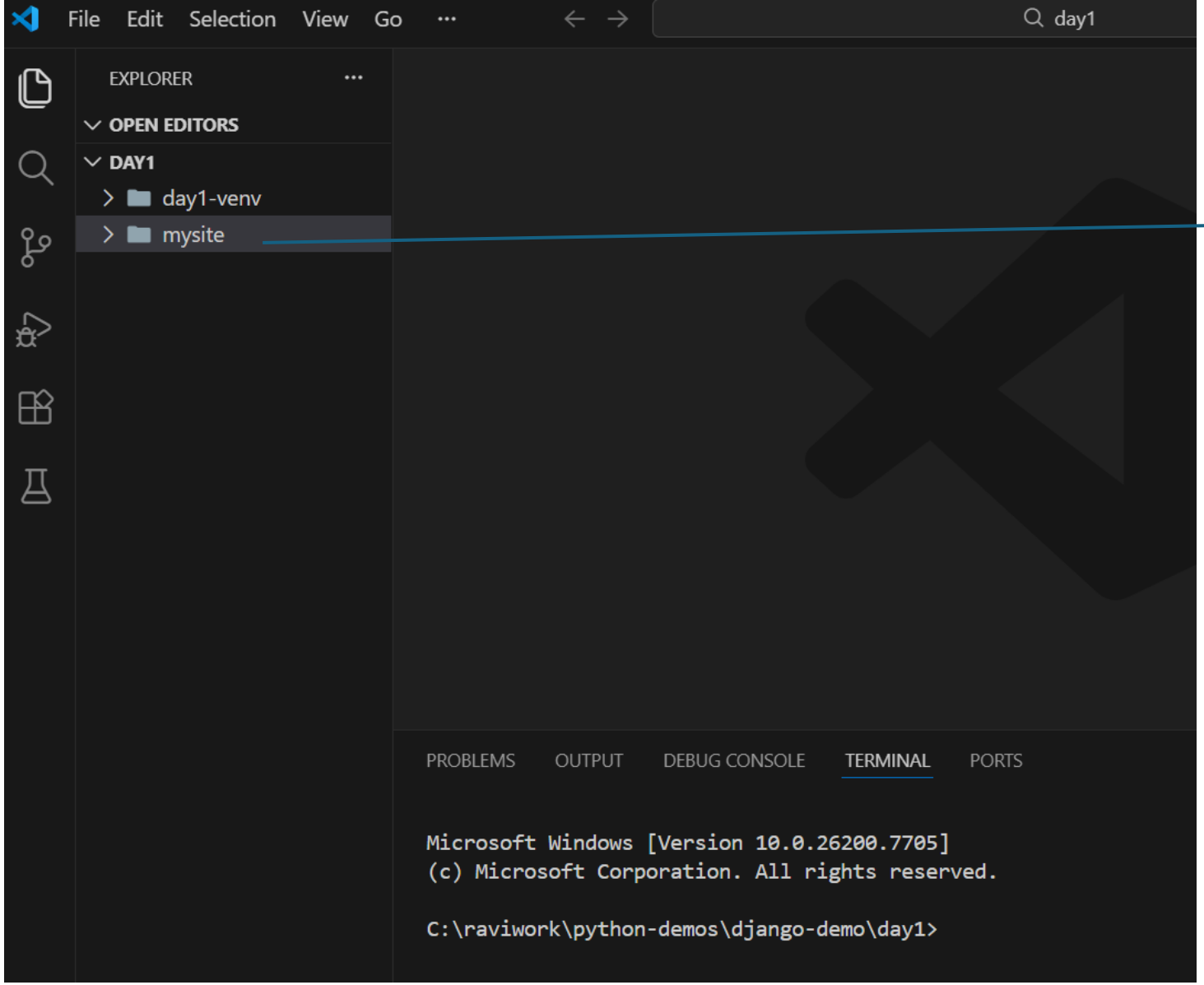
3.Template: Contains static HTML mixed with Django's templating syntax. Templates are used to display dynamic data on the web page.

Create a Project in Django

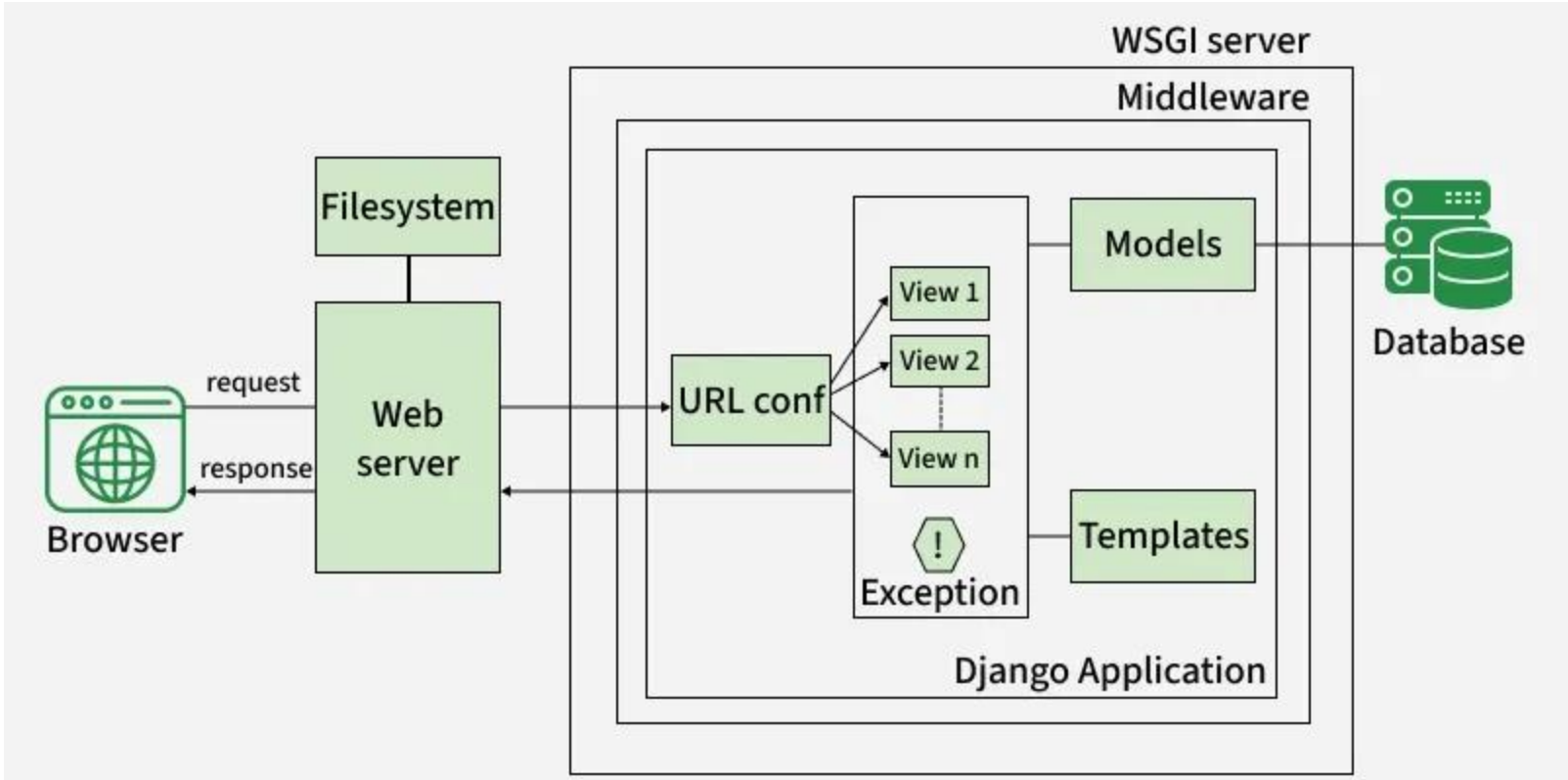
- A Django project is the main folder structure for a web application, containing settings, URLs, and apps.
- It sets up the environment for development.

Step 1: Create the Project

- Once the environment is activated and Django is available, a new project can be created using:
  - *(venv) C:\raviwork\python-demos\django-demo>django-admin startproject mysite*
  - This will create a new folder named **mysite.**
- Change into your project directory:
  - *cd mysite*



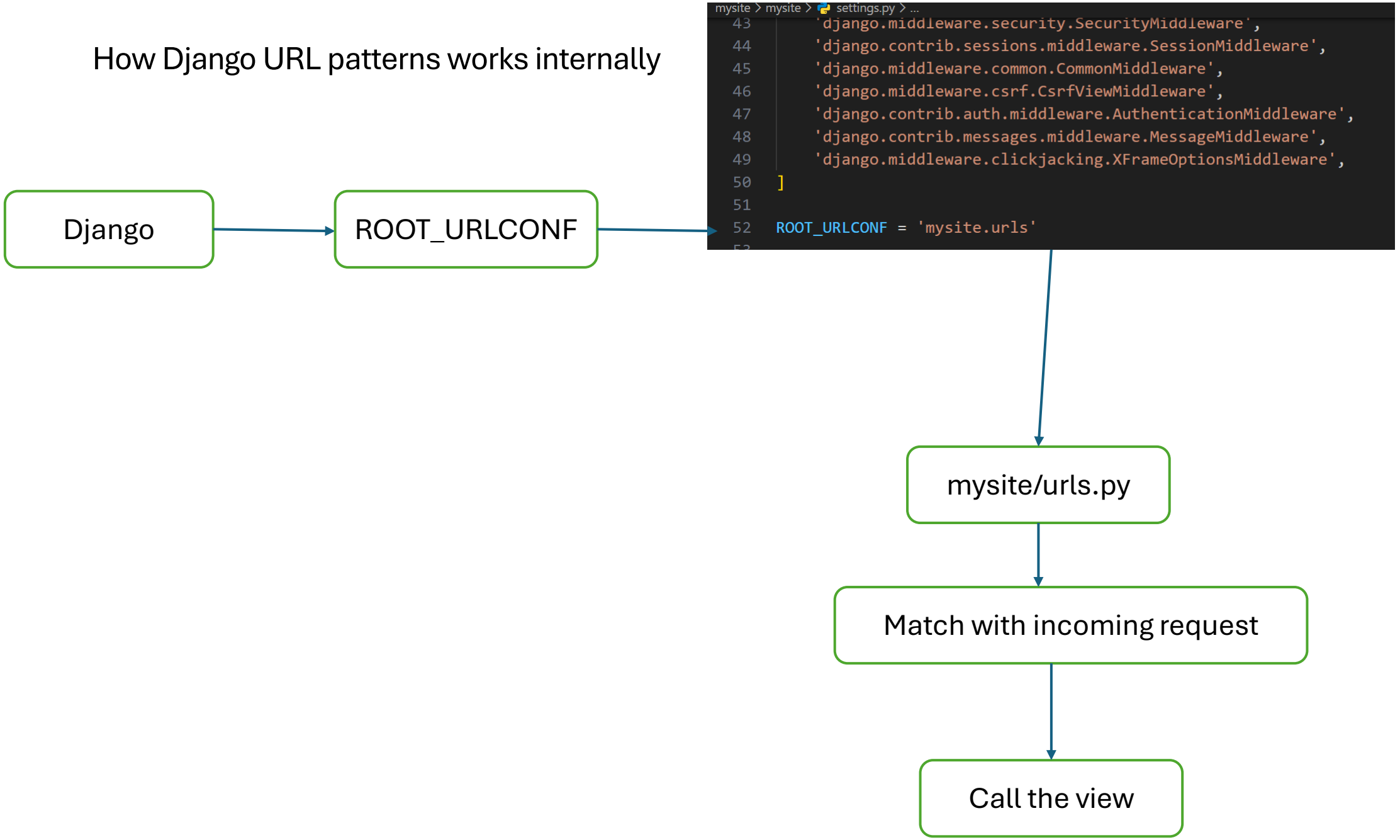
is root django project directroy


WSGI server
Middleware
Filesystem
Models
Database
request
response
Web server
URL conf
View 1
View 2
View n
Browser
Exception
Templates
Django Application

Installing Django

- *pip install django*

How Django URL patterns works internally



http://127.0.0.1:8000/myapp/

ROOT_URLCONF

settings.py
ROOT_URLCONF =
'mysite.urls'

mysite > myapp > urls.py > ...

```python
from django.urls import path
from . import views
urlpatterns = [
    path('',views.index),
]
```

urlpatterns = [
path('admin/', admin.site.urls),
path('myapp/',include('myapp.urls'))
]

mysite.urls

mysite > myapp > views.py > ...

```python
from django.shortcuts import render
from django.http import HttpResponse
# Create your views here.
def index(request):
    return HttpResponse("Welcome to Django World!")

```

Django Project Structure:

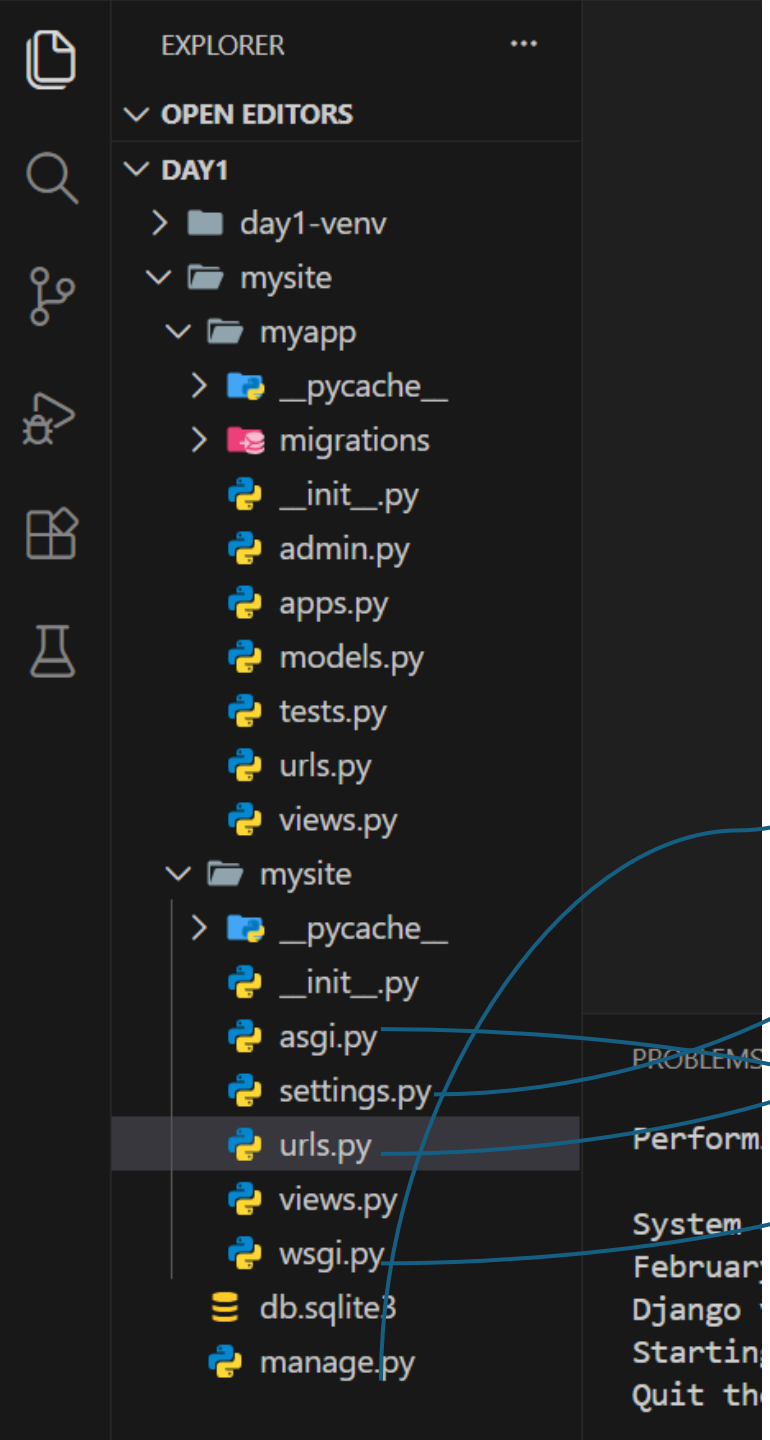
- Initiating a Django project with a simple console command:
  - >django-admin startproject my_project
- This command generates a skeletal framework with crucial configurations and settings:

```
my_project/
 ├── manage.py
 ├── my_project/
|  ├── __init__.py
|  ├── asgi.py
|  ├– settings.py
|  ├– urls.py
|  ├── wsgi.py
```

To create new Django app – python manage.py startapp my_app

Applications stand as self-contained entities, housing their own models, views, and templates.

```
my_project/
├── manage.py
├── my_project/
│   ├── __init__.py
│   ├── asgi.py
│   ├– settings.py
│   ├– urls.py
│   ├── wsgi.py
├── my_app/
    ├── migrations/
    │   └── __init__.py
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── tests.py
    ├── urls.py
    └── views.py
```



manage.py - serves as the gateway to various Django management commands. It's the tool through which you initiate the development server, create applications, run migrations, and more.

This file houses the settings that configure your Django project.
From database configurations to middleware lists,
this is where you define how your application functions.

maps URLs to views.
This file determines which view is displayed when a specific URL is accessed.

serves as the entry point for your application when deployed on a production server.

asgi.py is the entry point for asynchronous web servers.

Organizing Applications within the Project

Reusable Applications :

A reusable application is a self-contained package of code that encapsulates a specific functionality.

This functionality can range from simple utilities to complex features.

Django’s architecture encourages developers to create these applications so that they can be shared across projects without requiring extensive modifications.

Benefits of Reusable Applications:

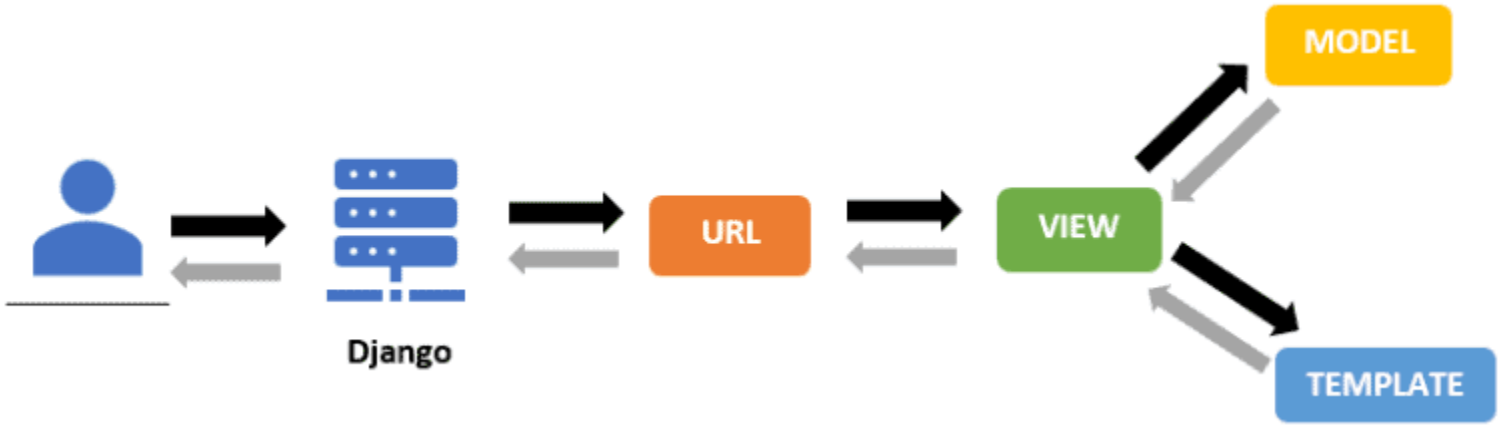
- Modularity:
- Code Reusability
- Standardization
- Maintainability
- Community contribution

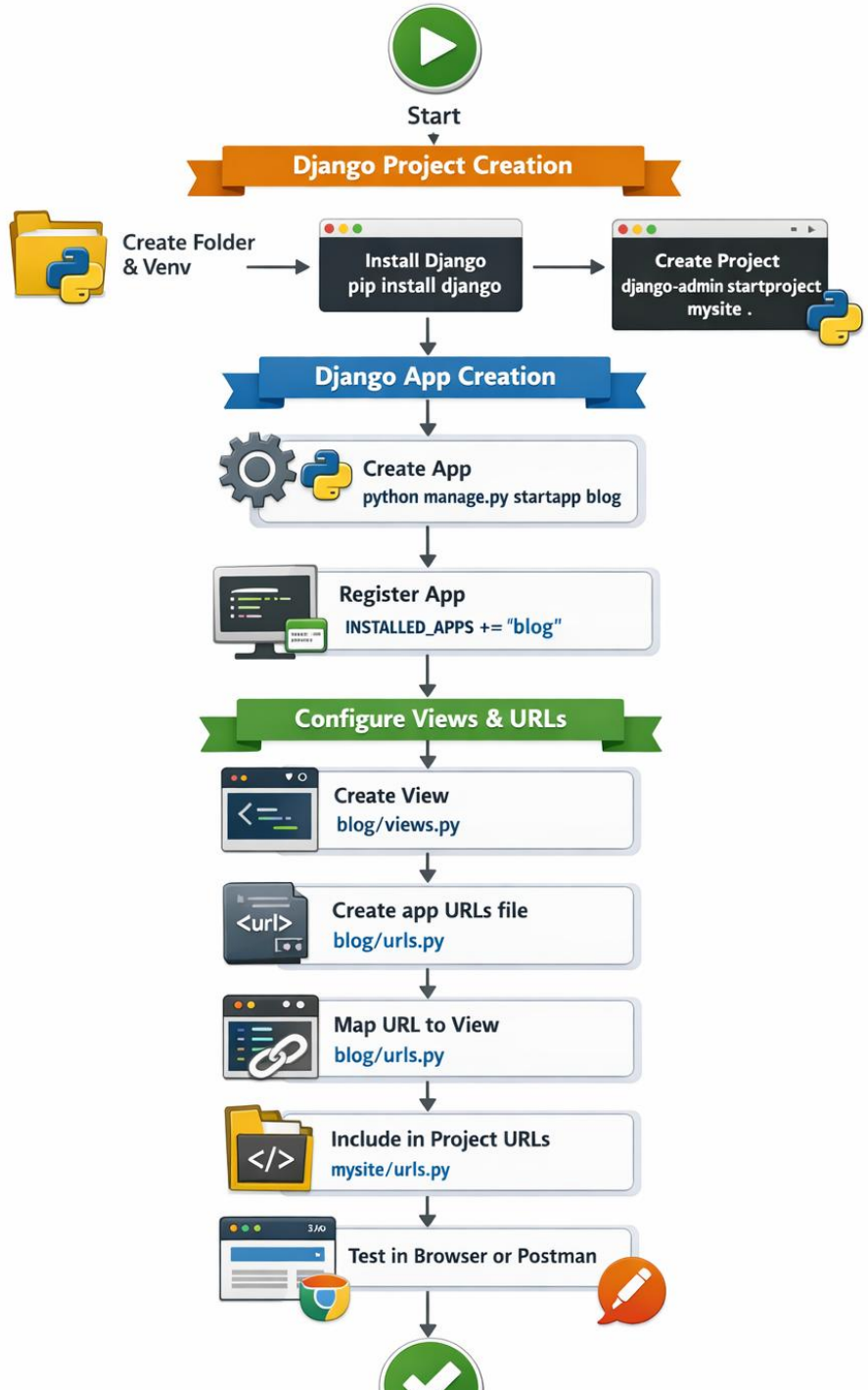
Best practice for project structure

- Maintaining a clean and organized project structure is crucial for the readability, scalability, and maintainability of your Django project.

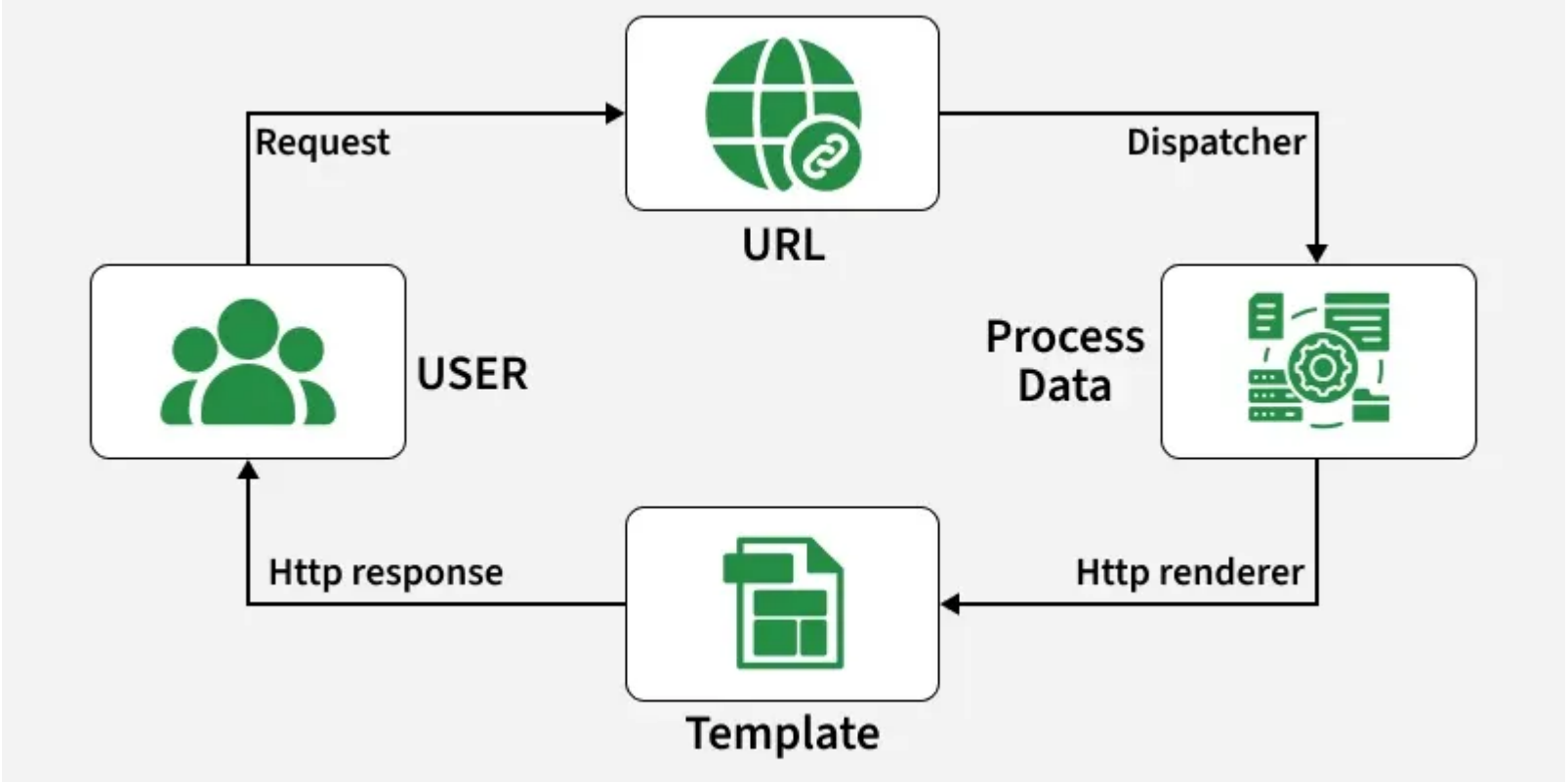
```
project_name/
 ├── manage.py
 ├── project_name/
|  ├── __init__.py
|  ├── asgi.py
|  ├– settings.py
|  ├– urls.py
|  ├── wsgi.py
 ├── app1/
 ├── app2/
...
 ├── static/
 ├── media/
 ├── templates/
```

Django workflow

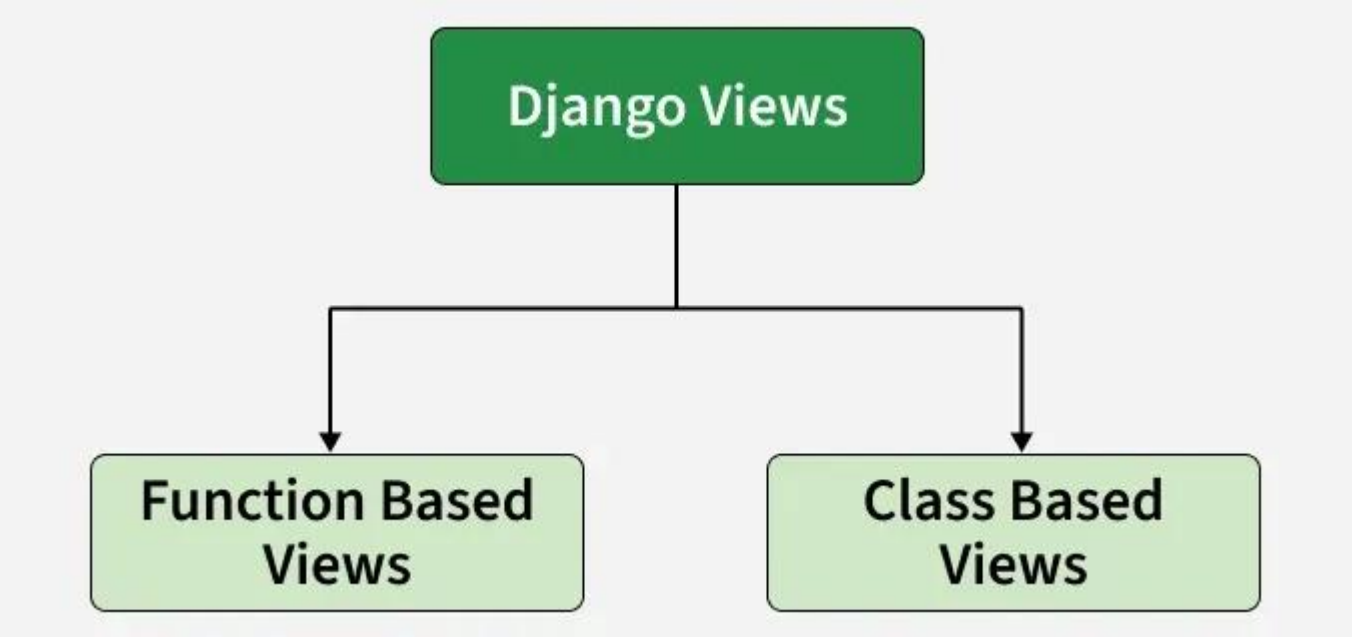



Start
Django Project Creation
Create Folder & Venv
Install Django
pip install django
Create Project
django-admin startproject mysite .
Django App Creation
Create App
python manage.py startapp blog
Register App
INSTALLED_APPS += "blog"
Configure Views & URLs
Create View
blog/views.py
Create app URLs file
blog/urls.py
Map URL to View
blog/urls.py
Include in Project URLs
mysite/urls.py
Test in Browser or Postman

Views In Django

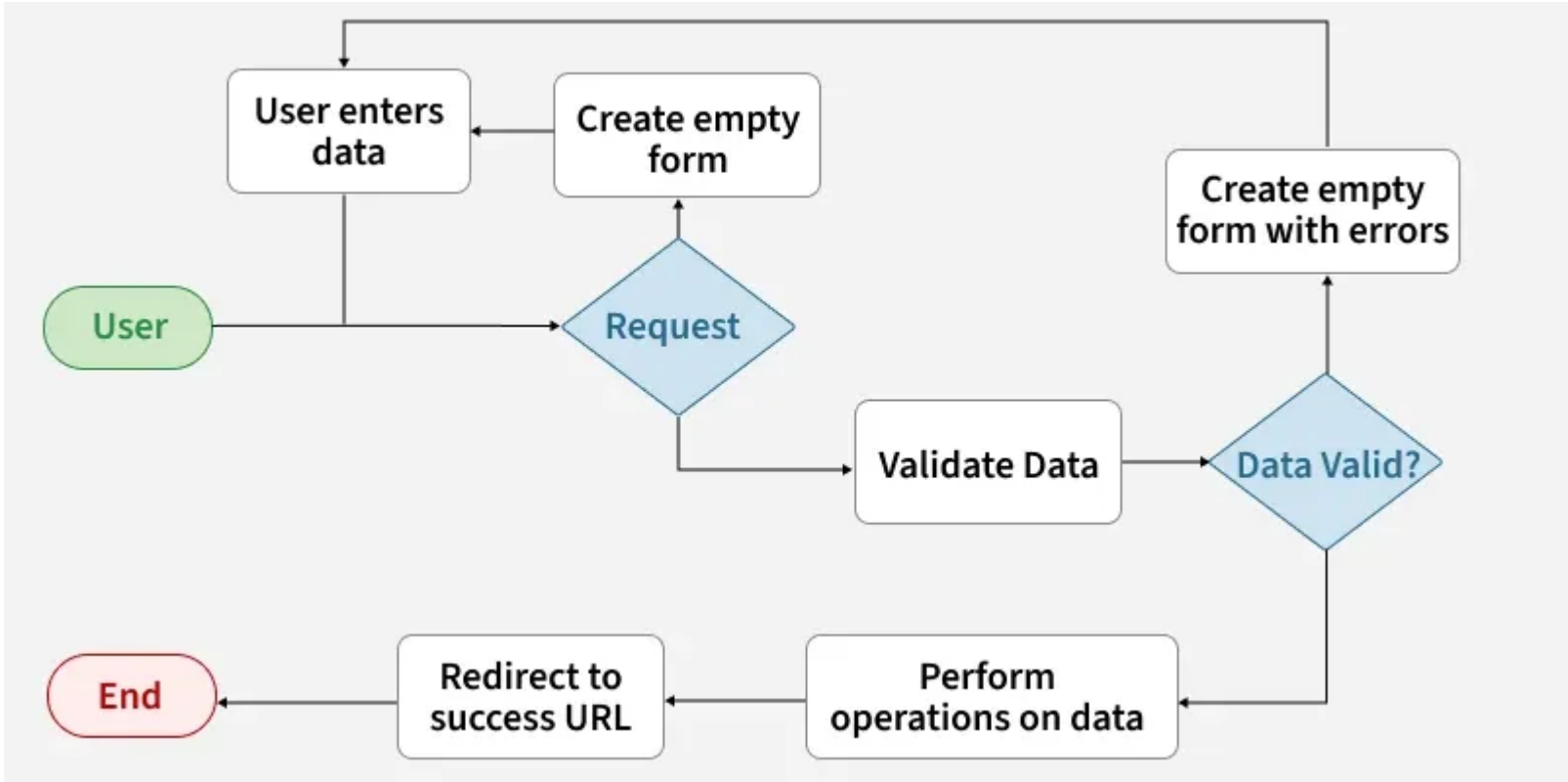


Django View Types



Django Forms

- Django Forms are used to collect input from users, check if the input is correct, and process the data.
  - Gather information through form fields such as text, email, or password.
  - Automatically validate the data before processing.
  - Can save the cleaned data to the database when needed.
  - Provide a clean way to handle user input in web applications.


User enters data
Create empty form
Create empty form with errors
User
Request
Validate Data
Data Valid?
End
Redirect to success URL
Perform operations on data

- **Syntax**
- *field_name = forms.FieldType(**options)*
  - **field_name**: Name of the form field.
  - **FieldType**: Type of the field (e.g., CharField, EmailField, IntegerField).
  - **options**: Optional keyword arguments such as max_length, required, initial, label, etc.

Creating a Simple Django Form

- To create a form, create a “forms.py” file in app folder:

```
from django import forms

class UserForm(forms.Form):
    first_name = forms.CharField(max_length=200)
    last_name = forms.CharField(max_length=200)
    roll_number = forms.IntegerField(help_text="Enter 6 digit roll number")
    password = forms.CharField(widget=forms.PasswordInput())
```

Render Django Forms

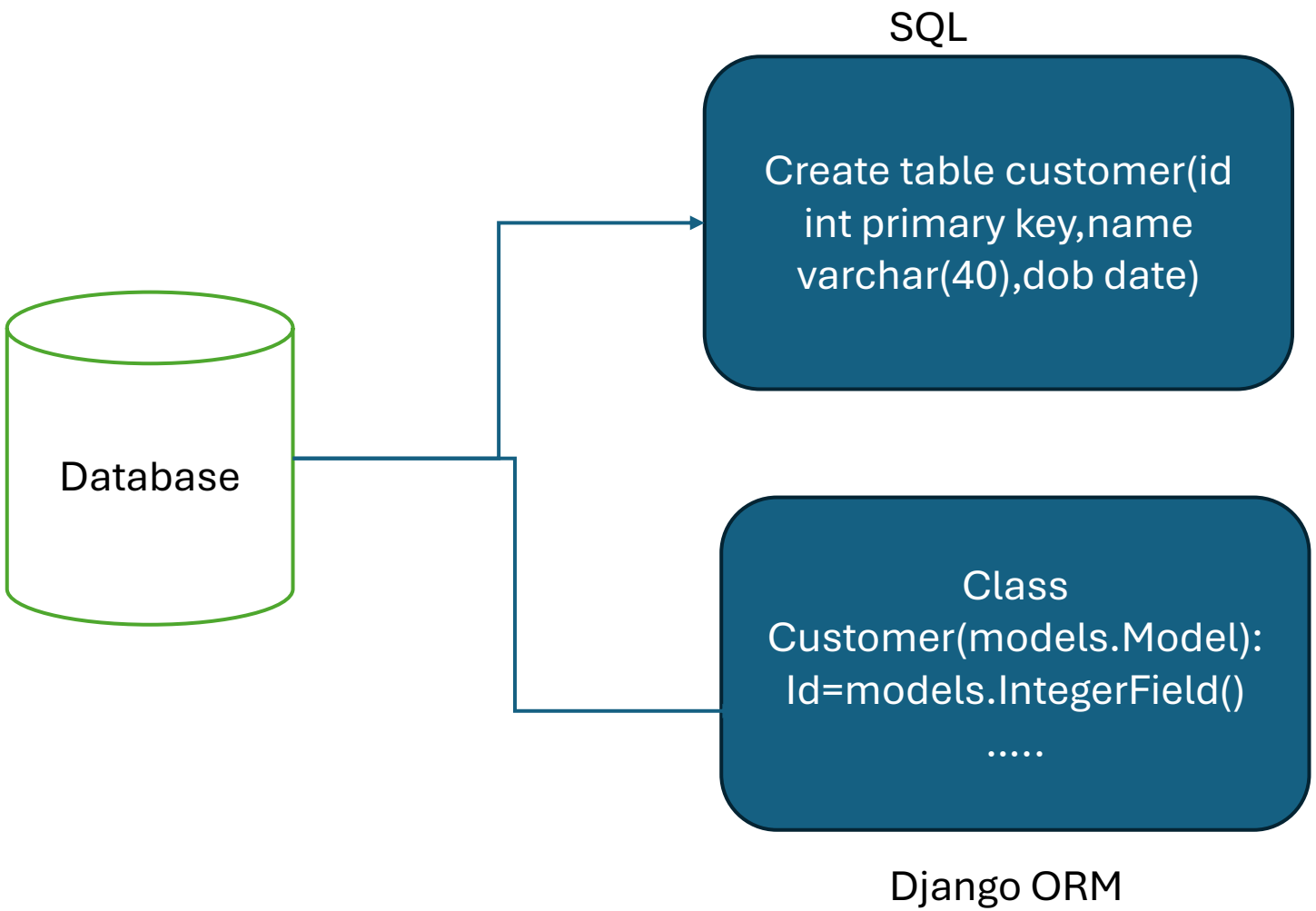
A form comes with 3 in-built methods that can be used to render Django form fields.

- {{ form.as_table }} will render them as table cells wrapped in <tr> tags
- {{ form.as_p }} will render them wrapped in <p> tags
- {{ form.as_ul }} will render them wrapped in <li> tags

Django Models

A Django model is a Python class that represents a database table. It lets developers manage data easily using Python instead of writing SQL.

- Django’s ORM provides an easy way to connect Python code with the database.
- Models help create, update, and delete data in a simple manner.
- It works smoothly with the Django admin panel for managing information.
- It supports data checks and table relationships using fields like ForeignKey and ManyToManyField.


SQL
Create table customer(id int primary key,name varchar(40),dob date)
Database
Class Customer(models.Model):
Id=models.IntegerField()
.....
Django ORM

Steps to create Django Model

- Create Django Model in models.py
- **Applying Migrations**
  - *python manage.py makemigrations*
    python manage.py migrate

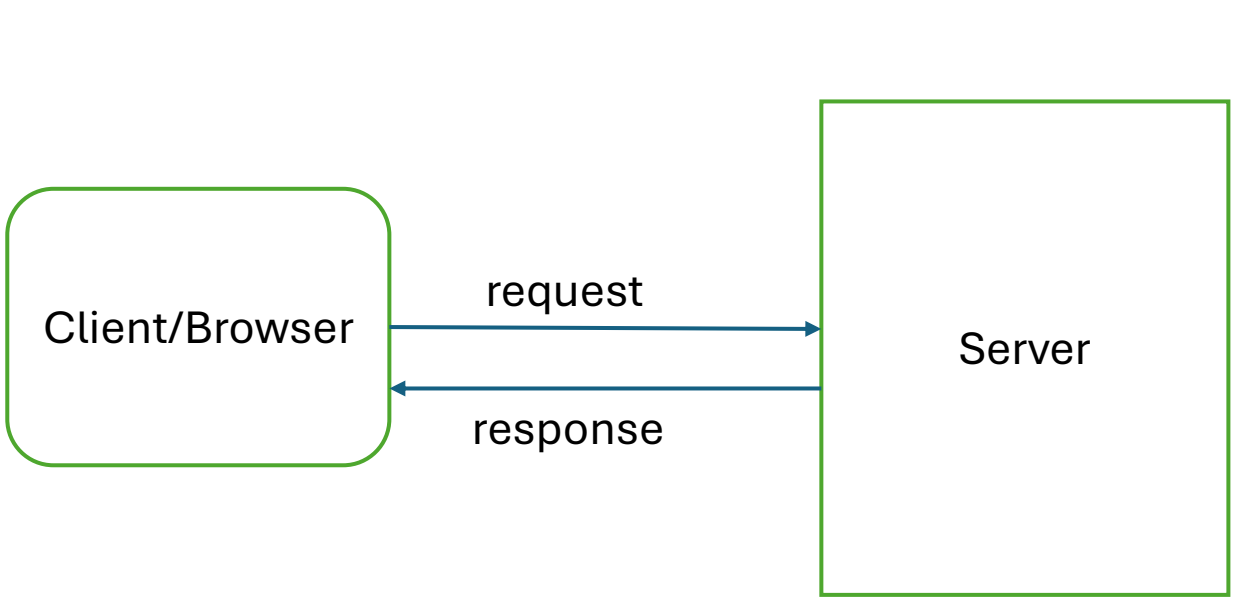
- **Registering Models in Django Admin**
  - To manage your model via the admin interface:
  - Open admin.py in app directory.
  - Register the model:

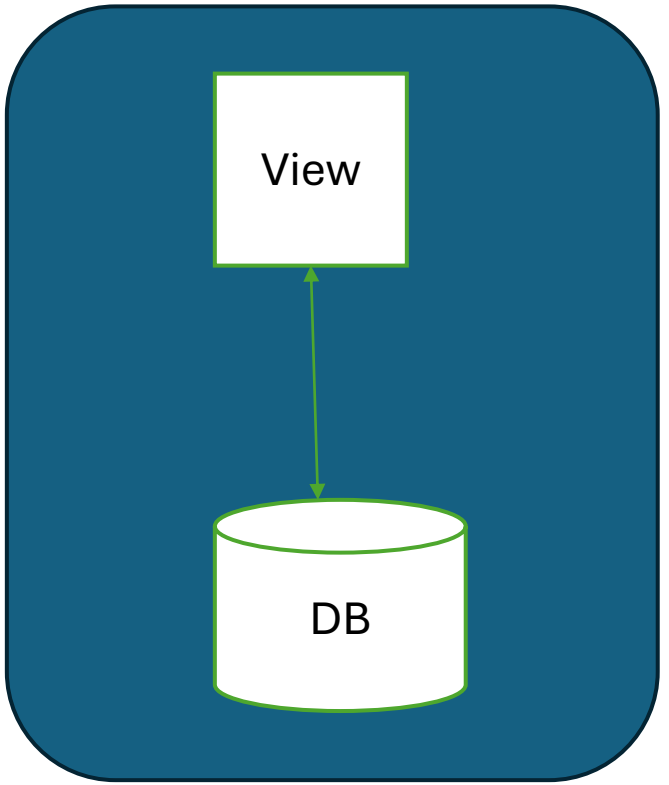
```
from django.contrib import admin

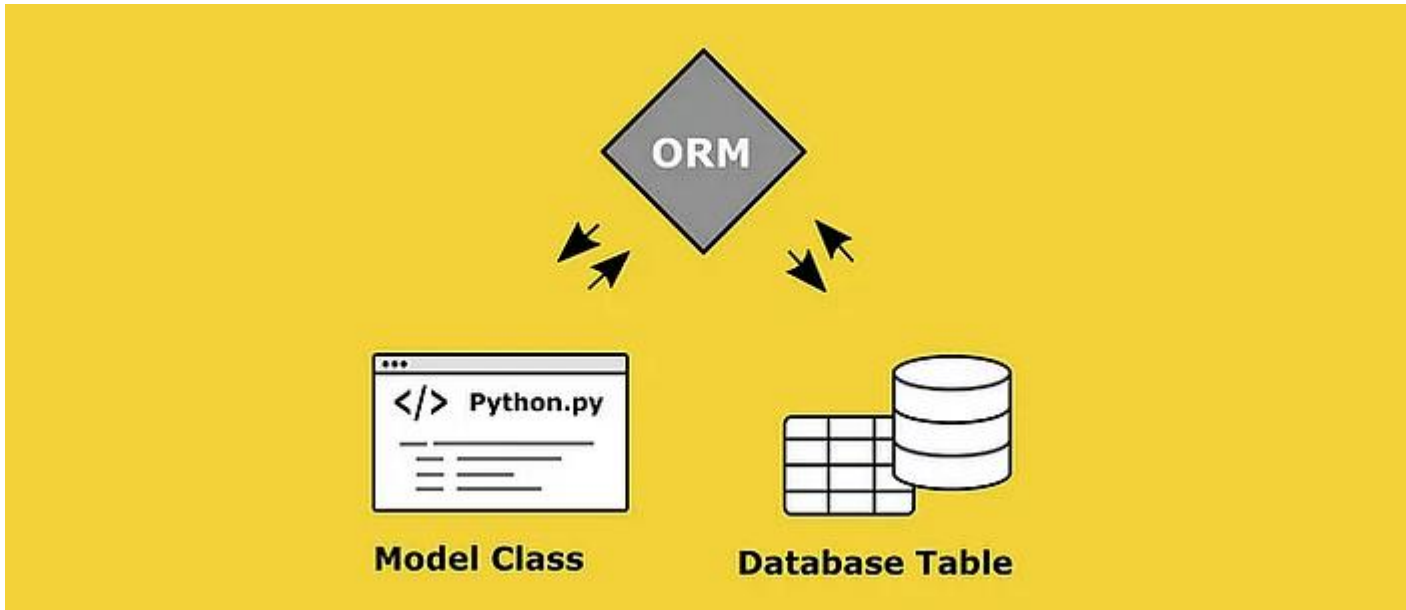
# Register your models here.
from .models import CustomerModel

admin.site.register(CustomerModel)
```

Databases in Django






ORM
</> Python.py
Model Class
Database Table


django
REST
framework

What is a REST API?

REST stands for Representational State Transfer. It is an architectural style for creating networked applications that rely on stateless, client-server communication.

REST APIs allow clients to interact with servers by making requests over HTTP, and these requests typically involve the following:

- **GET**: Retrieve data from the server.
- **POST**: Send data to the server to create a new resource.
- **PUT**: Update an existing resource on the server.
- **DELETE**: Remove a resource from the server.

What is Django REST Framework?

- Django REST Framework (DRF) is a powerful toolkit for building Web APIs in Django.
- It offers an easy-to-use interface to build REST APIs and comes with many features such as authentication, serialization, and viewsets that make developing APIs easier.

Setting up a Django Project for REST API

- pip install django djangorestframework
- django-admin startproject restapi_project
- python manage.py startapp restapi

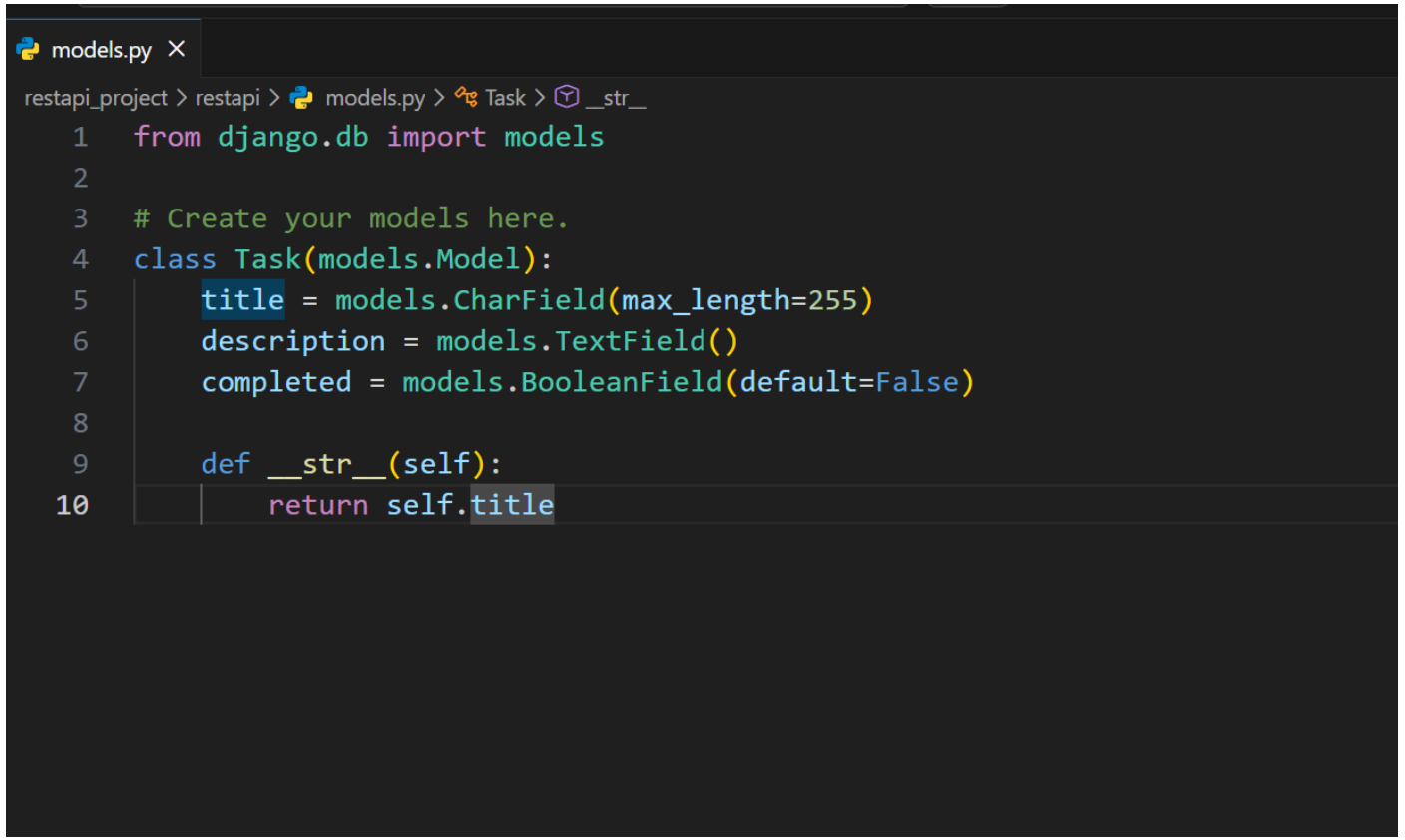
Set Up Django REST Framework

In your restapi_project/settings.py, add 'rest_framework' to the INSTALLED_APPS section:

```
INSTALLED_APPS = [
    # other installed apps,
    'rest_framework',
    'api',  # newly created app
]
```

Define Your Model

- In the api/models.py, define the Task model:



Run the following commands to create the necessary database tables:

- >python manage.py makemigrations
  >python manage.py migrate

Create a Serializer

- Serializers in DRF are responsible for converting complex data types like Django models into JSON, XML, or other content types, and vice versa.

Create Views and URLs

- In rest-api/views.py
- Use APIView or ViewSets to manage CRUD operations.

Define URLs

- Next, create URLs to map the views.
- In restapi/urls.py, define the API routes:

Include these URLs in your project’s main urls.py:

```
from django.urls import path
from . import views


urlpatterns = [
    path('tasks/', views.task_list),
    path('tasks/<int:pk>/', views.task_detail),
]
```